



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



Tecnológico Nacional de México

Instituto Tecnológico de Tijuana

Subdirección Académica

Departamento de Sistemas y Computación

Semestre: Agosto a Diciembre 2026

Carrera: Ingeniería en Sistemas Computacionales

Asignatura: DESARROLLO DE SOFTWARE CON APLICACIÓN EN LA NUBE

Unidad 2

Manual Técnico de Arquitectura del Proyecto Integrador

Docente: RAMON LOAIZA CHAVEZ

Alumnos:

Javier Ulises Cortés Aguilar 22211541

Brisa Julianna Aguayo Virgen 22211514

Fecha de entrega: 28/septiembre/2026

Índice

Introducción.....	3
Arquitectura de la solución.....	4
Servicios de Azure.....	4
Prerrequisitos.....	5
Herramientas necesarias.....	5
Componentes necesarios.....	6
Instalación de herramientas.....	6
Cómo comprobar que Docker funciona.....	7
Comprobar el estado de la suscripción de Azure.....	8
Control de versiones.....	9
IDE.....	9
Ambientes.....	10
Variables de entorno.....	12
Variables de entorno del backend.....	12
Variables de entorno del frontend.....	12
Dependencias.....	13
Dependencias del backend.....	13
Dependencias del frontend.....	13
Instalación en desarrollo.....	14
Clonación del repositorio.....	14
Base de datos de desarrollo con SQL Server en Docker.....	14
Protección de archivos sensibles con .gitignore.....	15
Aprovisionamiento en Azure.....	18
Registro de proveedores de recursos.....	18
Grupo de recursos.....	18
Servidor y base de datos de Azure SQL.....	18
Plan de App Service.....	19
Aplicación web (backend).....	19
App Settings de producción.....	19
Regla de firewall del servidor SQL.....	19
Static Web App (frontend).....	20
Verificación de los recursos provisionados.....	20
Build y despliegue.....	21
Scripts de ejecución del backend.....	21
Compilación en el servidor (SCM_DO_BUILD_DURING_DEPLOYMENT).....	21
Empaquetado del backend.....	21
Envío del backend.....	22
Verificación del backend en producción.....	22
Compilación del frontend para producción.....	23
Despliegue manual del frontend en Static Web Apps.....	23
Verificación de la aplicación en producción.....	23
Habilitar la autenticación básica de publicación en App Service.....	24
Configuración de credenciales de despliegue (GitHub Secrets).....	25
Pipeline de CI/CD del frontend (GitHub Actions).....	26
Pipeline de CI/CD del backend (GitHub Actions).....	28

Introducción

Este documento describe la arquitectura, los prerequisites, la configuración y el procedimiento de despliegue de CleanLink, una plataforma que conecta empresas con proveedores de limpieza. Está dirigido a desarrolladores, ingenieros DevOps y administradores de sistemas que necesiten instalar, desplegar y operar la solución. Cubre el ambiente de desarrollo (local, con SQL Server en Docker) y el de producción (Azure App Service, Static Web Apps y Azure SQL Database). La versión actual es un esqueleto funcional; la lógica de negocio y el modelo de datos multi-tenant se documentarán conforme se implementen.

Arquitectura de la solución

CleanLink es una aplicación web de tres capas: una SPA (React + Vite), una API REST (Node.js + Express) y una base de datos relacional (SQL Server en desarrollo, Azure SQL Database en producción). El frontend y el backend se despliegan de forma independiente; la SPA consume la API por HTTPS mediante la URL definida en VITE_API_URL, y la API accede a la base de datos con el driver mssql.

El aislamiento de datos entre clientes se diseñó como multi-tenant por esquema: cada tenant dispone de un esquema propio dentro de una misma base de datos. Esta característica está planeada y aún no implementada; la versión actual es un esqueleto que expone los endpoints /api/health y /api/db-health.

El despliegue no está containerizado: ambas capas se publican en servicios administrados de Azure (PaaS). Docker se usa únicamente en desarrollo para ejecutar SQL Server. Todos los recursos residen en el grupo rg-cleanlink.

Servicios de Azure

Capa	Servicio	Recurso	Región	Función
Frontend	Azure Static Web Apps	swa-cleanlink	centralus	Hosting de la SPA compilada.
Backend	Azure App Service (Linux, Node.js 24 LTS)	app-cleanlink-juca1010	westus	Ejecuta la API REST.
Backend	App Service Plan	plan-cleanlink	westus	Cómputo del App Service (plan gratuito).
Datos	Azure SQL Server	your_sql_server_name	westus	Servidor lógico: autenticación y firewall.
Datos	Azure SQL Database	cleanlinkdb	westus	Base de datos de la aplicación.

Prerrequisitos

Herramientas necesarias

Herramienta	Versión	Para qué la usamos en CleanLink
Node.js	v24.15.0 (LTS)	Ejecuta el backend (Express) y las herramientas de build del frontend (React).
npm	11.12.1	Gestor de paquetes: instala las dependencias de ambos proyectos y ejecuta scripts como <code>npm run build</code> .
Git	2.54.0	Control de versiones del código y base del despliegue automático (CI/CD).
Visual Studio Code	1.121.0	IDE recomendado para escribir y depurar el código.
WSL 2	2.7.14	Capa de Linux sobre la que corre Docker Desktop en Windows. Debe instalarse antes que Docker Desktop, con <code>wsl --install</code> (requiere reiniciar el equipo).
Docker	29.8.0	Contenerizar SQL Server en el ambiente de desarrollo, sin instalarlo directamente en la máquina.
Azure CLI	2.90.0	Crear y administrar los recursos de Azure desde la terminal, de forma reproducible.

Componentes necesarios

Componente	Requisito	Notas
Sistema operativo	Windows 10 22H2 o Windows 11 (64 bits)	Ambiente de desarrollo.
Gestor de instalación	winget	Incluido en Windows 11; en Windows 10 lo provee App Installer
Procesador	64 bits con virtualización activada	Necesaria para Docker Desktop (WSL 2).
Memoria RAM	8 GB recomendados (4 GB mínimo)	SQL Server en contenedor requiere al menos 2 GB por sí solo.
Almacenamiento	20 GB libres (recomendado)	Para Docker, imágenes, dependencias y el proyecto.
Suscripción de nube	En nuestro caso Azure for Students activa	Donde se crean los recursos del ambiente de producción.
Conexión a internet	Requerida	Para instalar paquetes y acceder al portal de Azure.

Instalación de herramientas

WSL 2 se instala primero, desde una terminal con permisos de administrador; requiere reiniciar el equipo. El resto de las herramientas se instala con winget.

Powershell

```
wsl --install --no-distribution
```

Powershell

```
$apps = @(
    "OpenJS.NodeJS.LTS",
    "Git.Git",
    "Microsoft.VisualStudioCode",
    "Docker.DockerDesktop",
    "Microsoft.AzureCLI"
)

foreach ($app in $apps) {
    winget install --id $app -e --accept-package-agreements
    --accept-source-agreements
}
```

Winget instala la última versión disponible de cada herramienta; las versiones de la tabla son las validadas para esta solución. Para comprobar las versiones instaladas:

Powershell

```
node -v; npm -v; git --version; docker --version; az --version;
wsl --version
```

Cómo comprobar que Docker funciona

Una vez instalados WSL 2 y Docker Desktop, se debe abrir Docker Desktop y esperar a que el indicador inferior izquierdo muestra "Engine running". Esto confirma que el motor de contenedores está activo.

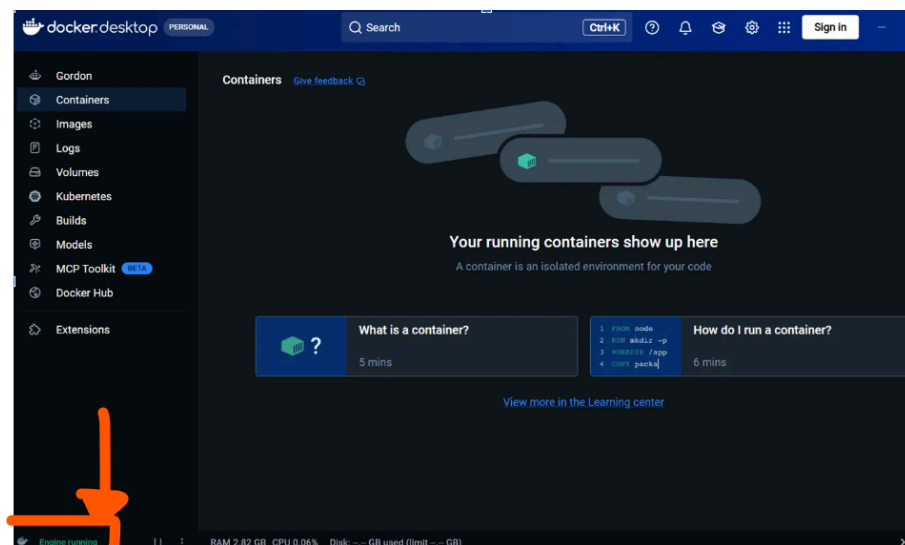


Figura 1. Docker Desktop en ejecución con el motor activo.

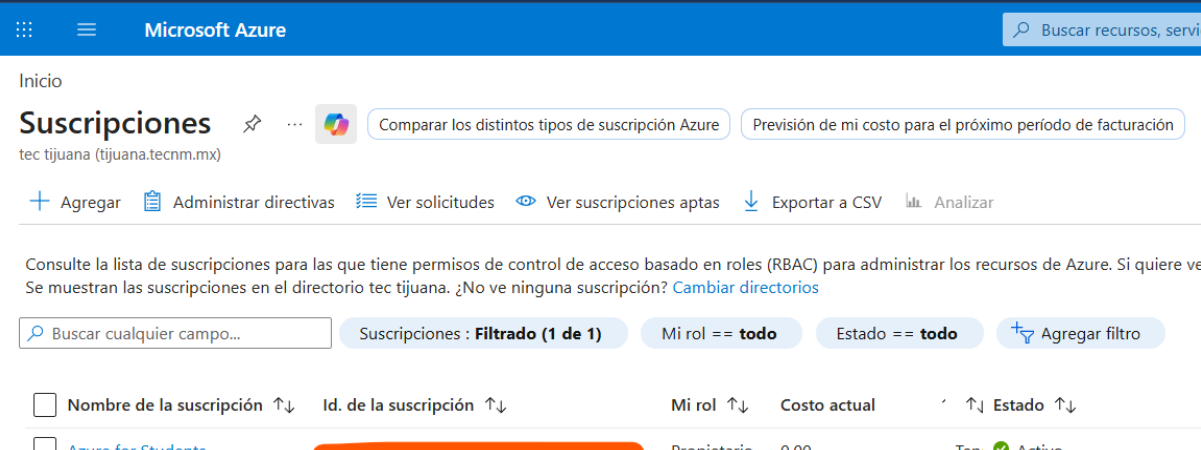
Para comprobar que el motor puede ejecutar contenedores, se ejecuta la imagen de prueba hello-world. Si la instalación es correcta, la terminal mostrará el mensaje "Hello from Docker!".

Powershell: Verificación del motor de Docker mediante el contenedor hello-world

```
PS C:\Windows\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
```

Comprobar el estado de la suscripción de Azure

Los recursos en la nube se crearán en una suscripción de Azure. En este proyecto se utiliza la suscripción Azure for Students, que debe aparecer con estado "Activo" en la sección Suscripciones del portal de Azure.



The screenshot shows the Microsoft Azure portal interface. At the top, there's a blue header with the Microsoft Azure logo and a search bar. Below the header, the 'Suscripciones' (Subscriptions) section is active. It shows a list of subscriptions with columns for 'Nombre de la suscripción', 'Id. de la suscripción', 'Mi rol', 'Costo actual', and 'Estado'. The first subscription listed is 'Azure for Students', which is marked as 'Activo' (Active) and has a cost of '0.00'. The page also includes a search bar, filters, and a table of subscriptions.

Nombre de la suscripción	Id. de la suscripción	Mi rol	Costo actual	Estado
Azure for Students		Propietario	0.00	Activo

Figura 2. Suscripción Azure for Students activa en el portal de Azure.

Para que Azure CLI pueda crear y administrar recursos en la suscripción del proyecto, se debe iniciar sesión con la cuenta institucional. El comando abre el navegador para autenticarse y, al terminar, la terminal muestra el tenant y la suscripción activa.

Powershell: Azure CLI autenticada con la suscripción Azure for Students.

```
PS C:\Windows\system32> az login
Select the account you want to log in with. For more information on login with
Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136
Retrieving tenants and subscriptions for the selection...
[Tenant and subscription selection]
No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure for Students    id-de-suscripcion   tec tijuana
The default is marked with an *; the default tenant is 'tec tijuana' and
subscription is 'Azure for Students' (id-de-suscripcion).
Select a subscription and tenant (Type a number or Enter for no changes):
Tenant: tec tijuana
Subscription: Azure for Students (id-de-suscripcion)
[Announcements]
With the new Azure CLI login experience, you can select the subscription you
want to use more easily. Learn more about it and its configuration at
https://go.microsoft.com/fwlink/?linkid=2271236
If you encounter any problem, please open an issue at https://aka.ms/azclibug
```

Control de versiones

Se configura Git con la identidad del desarrollador antes del primer commit, hay que usar main como rama principal y ajustar el manejo de saltos de línea en Windows.

Powershell: configuración de git

```
git config --global user.name "your_name_here"
git config --global user.email "your_email_here"
git config --global init.defaultBranch main
git config --global core.autocrlf true
```

IDE

Usaremos Visual Studio Code, y las siguientes extensiones:

Extensión	Identificador	Para qué sirve
ESLint	dbaeumer.vscode-eslint	Detecta errores y malas prácticas en el código JavaScript.
Prettier	esbenp.prettier-vscode	Da formato uniforme al código del equipo.

Extensión	Identificador	Para qué sirve
Docker	ms-azuretools.vscode-docker	Administra imágenes y contenedores desde el editor.
SQL Server (mssql)	ms-mssql.mssql	Permite conectarse a SQL Server y ejecutar consultas.
Azure App Service	ms-azuretools.vscode-azureappservice	Consulta y administra el App Service desde VS Code.
GitHub Actions	github.vscode-github-actions	Muestra el estado de los pipelines de CI/CD.

Ambientes

Aspecto	Desarrollo	Producción
Frontend	Servidor de Vite en localhost:5173	Azure Static Web Apps (centralus)
Backend	node index.js en localhost:3000	Azure App Service (westus, plan gratuito)
Base de datos	SQL Server 2022 en contenedor Docker	Azure SQL Database (cleanlinkdb, westus)
Sistema operativo	Windows 10 22H2 / Windows 11	Linux (App Service)
Node.js	v24.15.0 LTS	NODE 24-lts
Docker	Solo para la base de datos	No se usa (despliegue sin contenerizar)
Variables de entorno	Archivos .env locales, fuera del repositorio	App Settings de App Service; VITE_API_URL se define en el pipeline
Despliegue	Manual	CI/CD con GitHub Actions al hacer push a main

Aspecto	Desarrollo	Producción
Certificado de la BD	Autofirmado (DB_TRUST_CERT=true)	Válido (DB_TRUST_CERT=false)

El desarrollo ocurre en Windows y la producción corre en Linux. `core.autocrlf true` normaliza los saltos de línea al hacer commit, y los nombres de archivo en Linux distinguen mayúsculas de minúsculas, por lo que deben respetarse en las importaciones.

Variables de entorno

Variables de entorno del backend

Variable	Descripción	Desarrollo	Producción
PORT	Puerto en el que escucha la API.	3000	Asignado por Azure App Service
DB_SERVER	Dirección del servidor de base de datos.	localhost	your_sql_server_name.database.windows.net
DB_PORT	Puerto de SQL Server.	1433	1433
DB_USER	Usuario de la base de datos.	sa	cleanlinkadmin
DB_PASSWORD	Contraseña del usuario de la base de datos.	your_secure_password_here	(guardada en App Settings de Azure App Service)
DB_NAME	Base de datos a la que se conecta la API.	master	cleanlinkdb
DB_TRUST_CERT	Acepta certificados autofirmados en la conexión.	true	false

Variables de entorno del frontend

Variable	Descripción	Desarrollo	Producción
VITE_API_URL	URL base de la API. Se inyecta en tiempo de build.	http://localhost:3000	https://your_app_name.azurewebsites.net

Dependencias

Dependencias del backend

Paquete	Versión	Función
express	5.2.1	Framework HTTP para la API REST.
mssql	12.7.2	Cliente de SQL Server / Azure SQL Database.
cors	2.8.6	Middleware de CORS para permitir peticiones desde el frontend.
dotenv	18.0.4	Carga las variables de entorno desde .env.

Dependencias del frontend

Paquete	Versión	Tipo	Función
react	^19.2.8	Producción	Librería de UI.
react-dom	^19.2.8	Producción	Renderizado de React en el navegador.
vite	^8.3.0	Desarrollo	Servidor de desarrollo y bundler de producción.
@vitejs/plugin-react	^6.1.1	Desarrollo	Soporte de React (JSX, Fast Refresh) en Vite.
eslint	^10.10.0	Desarrollo	Análisis estático del código.
@eslint/js	^10.0.1	Desarrollo	Reglas base recomendadas de ESLint.
eslint-plugin-react-hooks	^7.1.1	Desarrollo	Reglas para el uso correcto de hooks.
eslint-plugin-react-refresh	^0.5.6	Desarrollo	Válida componentes compatibles con Fast Refresh.
globals	^17.12.0	Desarrollo	Variables globales del entorno para ESLint.
@types/react, @types/react-dom	^19.2.18, ^19.2.7	Desarrollo	Definiciones de tipos para autocompletado en el IDE.

Instalación en desarrollo

Clonación del repositorio

Todo el código de la solución (backend, frontend y workflows de CI/CD) vive en un repositorio de GitHub, por lo que el primer paso es clonarlo en la máquina de desarrollo. El comando `git clone` descarga el repositorio completo con su historial y crea la carpeta `cleanlink`. Se recomienda usar la ruta `C:\dev`, que es la que utilizan los comandos del resto del manual.

Powershell:

```
mkdir C:\dev
cd C:\dev
git clone https://github.com/CortesAguilar/cleanlink.git
cd cleanlink
```

Para confirmar que la clonación fue correcta, `git status` debe indicar que la rama activa es `main` y que no hay cambios pendientes, y `git log --oneline` debe listar los commits del proyecto.

Powershell:

```
git status
git log --oneline
```

Base de datos de desarrollo con SQL Server en Docker

En el ambiente de desarrollo, la base de datos es una instancia de SQL Server 2022 que se ejecuta en un contenedor de Docker, lo que evita instalar el motor directamente en la máquina. El contenedor se crea con `docker run`, y la contraseña del usuario `sa` se define mediante la variable de entorno `MSSQL_SA_PASSWORD`. Debe tener al menos 8 caracteres e incluir mayúsculas, minúsculas, números y símbolos; de lo contrario, SQL Server no inicia.

Powershell:

```
docker run -d --name sql-cleanlink `
  -e "ACCEPT_EULA=Y" `
  -e "MSSQL_SA_PASSWORD=your_secure_password_here" `
  -p 1433:1433 `
  -v sqldata:/var/opt/mssql `
  mcr.microsoft.com/mssql/server:2022-latest
```

Los archivos `.env` no se incluyen en el repositorio. Para configurarlos, se copian las plantillas y se editan con los valores del ambiente.

PowerShell

```
copy backend\.env.example backend\.env  
copy frontend\.env.example frontend\.env
```

Para iniciar la API en el ambiente de desarrollo, primero se instalan sus dependencias con `npm install`, que lee el archivo `package.json` y las descarga en la carpeta `node_modules`, excluida del repositorio. Después se ejecuta el servidor de Node desde la carpeta `backend`; al arrancar, la terminal mostrará el puerto en el que escucha. Finalmente, el endpoint `/api/health` permite comprobar que el servidor responde correctamente.

Powershell:

```
cd C:\dev\cleanlink\backend  
npm install  
node index.js
```

Protección de archivos sensibles con `.gitignore`

Archivo: `.gitignore` (raíz del repositorio)

```
# Dependencias  
node_modules/  
  
# Variables de entorno (contienen secretos)  
.env  
.env.local  
  
# Archivos generados por el build  
dist/  
  
# Sistema y editor  
.DS_Store  
Thumbs.db
```

Antes de subir el código a un repositorio, se crea un archivo `.gitignore` en la raíz del proyecto. Este archivo indica a Git qué archivos debe ignorar, de modo que las dependencias, los archivos generados y, sobre todo, los archivos `.env` que contienen credenciales nunca se incluyan en el repositorio.

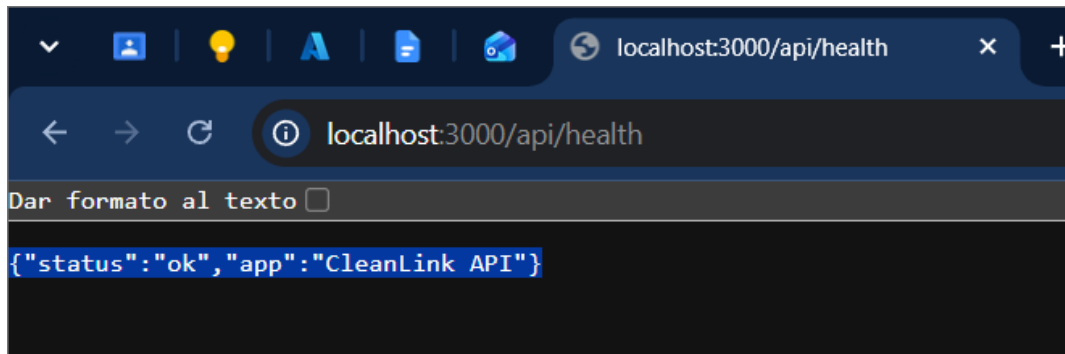


Figura 3. Respuesta del endpoint /api/health que confirma que la API de CleanLink está en ejecución.

Con el backend en ejecución y el contenedor de SQL Server activo, se verifica la conectividad accediendo al endpoint /api/db-health. Una respuesta con "database":"connected" confirma que la API se conecta correctamente a la base de datos.

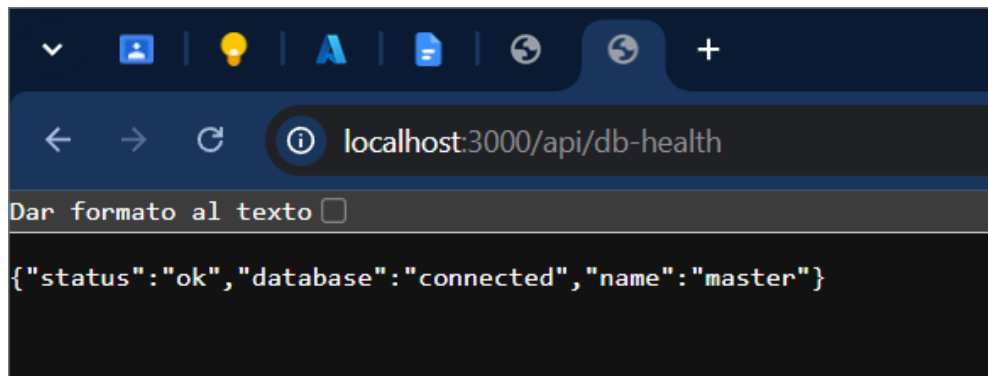


Figura 4. Respuesta del endpoint /api/db-health que confirma la conexión del backend con SQL Server en el ambiente de desarrollo.

Para descargar las dependencias del frontend, se ejecuta `npm install` desde la carpeta frontend. Este comando lee el archivo `package.json` e instala las librerías necesarias en la carpeta `node_modules`, que no se incluye en el repositorio.

Powershell:

```
cd frontend
npm install
```

Para iniciar el frontend en el ambiente de desarrollo, se ejecuta el servidor de desarrollo de Vite desde la carpeta frontend. Con el backend y el contenedor de SQL Server activos, la aplicación disponible en `http://localhost:5173` consulta ambos

endpoints y muestra su estado, lo que confirma la comunicación completa entre el frontend, la API y la base de datos.

Powershell:

```
cd C:\dev\cleanlink\frontend  
npm run dev
```



Figura 5. Pantalla inicial de CleanLink en el ambiente de desarrollo, con la API y la base de datos reportadas como conectadas.

Aprovisionamiento en Azure

Registro de proveedores de recursos

En una suscripción nueva, Azure no tiene activados todos los tipos de servicio. Antes de crear el primer recurso de cada tipo, se debe registrar su proveedor de recursos en la suscripción. Este paso se realiza una sola vez. Para CleanLink se registran Microsoft.Sql (Azure SQL Database) y Microsoft.Web (Azure App Service).

Powershell

```
az provider register --namespace Microsoft.Sql --wait  
az provider register --namespace Microsoft.Web --wait
```

Grupo de recursos

Todos los recursos de la solución se agrupan en un grupo de recursos, que permite administrarlos y eliminarlos como una unidad.

Powershell

```
az group create --name rg-cleanlink --location westus3
```

Servidor y base de datos de Azure SQL

El servidor lógico gestiona la autenticación y el firewall; su nombre debe ser único a nivel global, porque forma parte del endpoint <nombre>.database.windows.net. La base de datos se crea en el nivel Basic.

Powershell

```
az sql server create --name your_sql_server_name --resource-group  
rg-cleanlink --location westus --admin-user cleanlinkadmin  
--admin-password your_secure_password_here  
  
az sql db create --name cleanlinkdb --server your_sql_server_name  
--resource-group rg-cleanlink --edition Basic --service-objective  
Basic
```

Plan de App Service

El plan define el cómputo del backend. Se usa el SKU gratuito F1 sobre Linux. Antes de crear la aplicación, se verifica que el runtime de Node.js requerido esté disponible en la región.

Powershell:

```
az appservice plan create --name plan-cleanlink --resource-group rg-cleanlink --location westus --sku F1 --is-linux  
  
az webapp list-runtimes --os-type linux | Select-String NODE
```

Aplicación web (backend)

La aplicación se crea sobre el plan anterior con el runtime Node.js 24 LTS. El nombre debe ser único a nivel global, ya que define la URL pública <nombre>.azurewebsites.net.

Powershell

```
az webapp create --name your_app_name --resource-group rg-cleanlink --plan plan-cleanlink --runtime "NODE:24-lts"
```

App Settings de producción

Las variables de entorno del backend se definen como App Settings, que App Service inyecta como variables de entorno del proceso. PORT no se define, porque lo asigna la plataforma. Se usa --output none para que el comando no imprima los valores, incluida la contraseña.

Powershell

```
az webapp config appsettings set --name your_app_name --resource-group rg-cleanlink --output none --settings DB_SERVER=your_sql_server_name.database.windows.net DB_PORT=1433 DB_USER=cleanlinkadmin DB_PASSWORD=your_secure_password_here DB_NAME=cleanlinkdb DB_TRUST_CERT=false
```

Regla de firewall del servidor SQL

Por defecto, el servidor rechaza toda conexión entrante. La regla 0.0.0.0 – 0.0.0.0 habilita el acceso desde servicios de Azure, lo que permite que el App Service se conecte a la base de datos.

Powershell

```
az sql server firewall-rule create --server your_sql_server_name
--resource-group rg-cleanlink --name AllowAzureServices
--start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
```

Static Web App (frontend)

El frontend se aloja en Azure Static Web Apps, en el nivel gratuito. Se crea sin vincular un repositorio (no se usa --source), porque el despliegue se realiza desde el pipeline de GitHub Actions con un token, como se describe en la sección Build y despliegue. Static Web Apps solo está disponible en un subconjunto de regiones, por lo que puede quedar en una distinta a la del backend.

Powershell

```
az staticwebapp create --name swa-your_static_web_app_name
--resource-group rg-cleanlink --location centralus --sku Free
```

Verificación de los recursos provisionados

Una vez creados todos los recursos, se verifica en el portal de Azure que el grupo de recursos rg-cleanlink contiene los cinco componentes de la solución: el servidor y la base de datos de Azure SQL, el plan y la aplicación de Azure App Service (backend) y la Static Web App (frontend).

The screenshot shows the Azure portal interface for the resource group 'rg-cleanlink'. The 'Recursos' tab is selected, displaying a table of resources. The table has columns for 'Nombre', 'Tipo', and 'Ubicación'. The resources listed are:

Nombre	Tipo	Ubicación
app-cleanlink-juca1010	App Service	West US
cleanlinkdb (sql-cleanlink-juca1010/cleanlinkdb)	Base de datos SQL	West US
plan-cleanlink	Plan de App Service	West US
sql-cleanlink-juca1010	SQL Server	West US
swa-cleanlink	Aplicación web estática	Central US

Figura 6. Recursos de Azure que componen la solución CleanLink, agrupados en el grupo de recursos rg-cleanlink.

Build y despliegue

Scripts de ejecución del backend

Azure App Service inicia las aplicaciones de Node ejecutando npm start. Por eso el archivo package.json del backend define un script start que lanza el servidor. Es el mismo comando que se usa en desarrollo (node index.js), pero con un nombre estándar que cualquier plataforma de despliegue reconoce.

Archivo: backend/package.json (sección scripts)

```
"scripts": {  
  "start": "node index.js",  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

Una vez definido el script, la aplicación puede iniciarse en cualquier ambiente con el siguiente comando:

Powershell

```
npm start
```

Compilación en el servidor (SCM_DO_BUILD_DURING_DEPLOYMENT)

Esta App Setting hace que App Service ejecute npm install durante el despliegue, por lo que el paquete se envía sin node_modules.

Powershell:

```
az webapp config appsettings set --name your_app_name  
--resource-group rg-cleanlink --output none --settings  
SCM_DO_BUILD_DURING_DEPLOYMENT=true
```

Empaquetado del backend

El código se comprime en un archivo .zip, excluyendo node_modules y los archivos .env, que contienen credenciales.

Powershell

```
cd C:\dev\cleanlink\backend
Get-ChildItem -Exclude node_modules, .env, .env.* |
Compress-Archive -DestinationPath ..\backend.zip -Force
```

Envío del backend

El paquete se despliega en App Service. Este envío manual sirve para el primer despliegue; después lo automatiza el pipeline de GitHub Actions.

Powershell:

```
az webapp deploy --name your_app_name --resource-group
rg-cleanlink --src-path ..\backend.zip --type zip
```

Verificación del backend en producción

Una vez completado el despliegue, se verifica que la API responde en el ambiente de producción accediendo al endpoint `/api/health` desde la URL pública de la aplicación. Una respuesta con `"status":"ok"` confirma que el servicio se inició correctamente en Azure App Service. Con el plan gratuito, la primera solicitud tras un periodo de inactividad puede tardar hasta un minuto, porque la aplicación se reactiva.

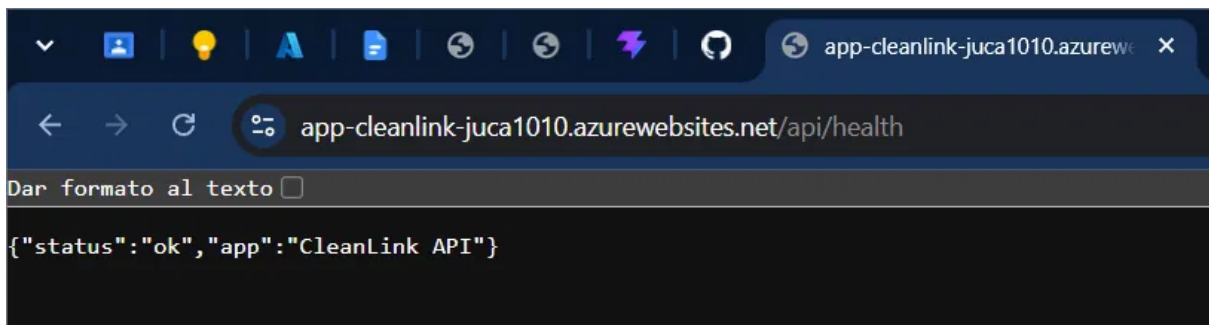


Figura 7. Respuesta del endpoint `/api/health` desde Azure App Service, que confirma que el backend de CleanLink se ejecuta en producción.

Para comprobar la conexión con la base de datos de producción, se consulta el endpoint `/api/db-health`. El valor `cleanlinkdb` en la respuesta confirma que la aplicación usa la base de datos de Azure SQL.

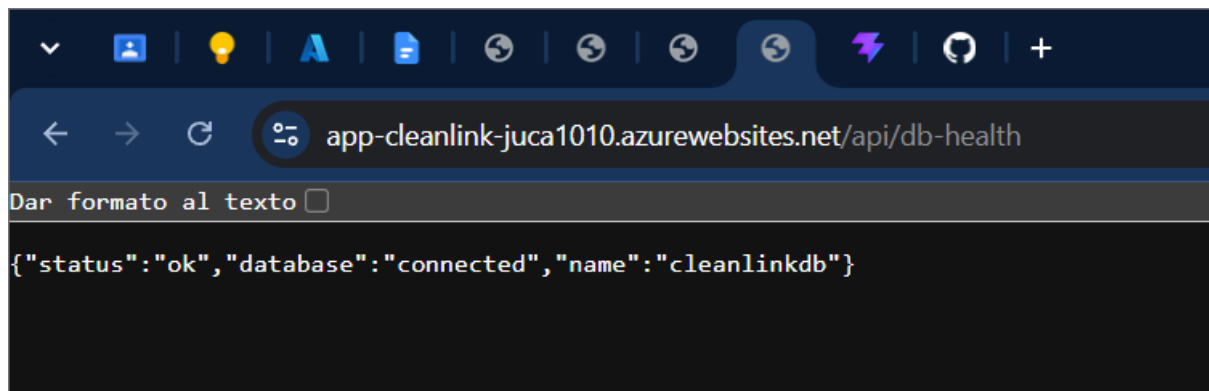


Figura 8. Respuesta del endpoint /api/db-health desde Azure App Service, que confirma la conexión del backend con Azure SQL Database en producción.

Compilación del frontend para producción

El frontend se compila con Vite, que genera en la carpeta dist los archivos estáticos optimizados (HTML, CSS y JavaScript) listos para publicar. Antes de compilar, se define la variable VITE_API_URL con la dirección del backend de producción.

Powershell

```
cd frontend
$env:VITE_API_URL = "https://your_app_name.azurewebsites.net"
npm run build
```

Despliegue manual del frontend en Static Web Apps

Los archivos compilados en dist se publican con la CLI de Static Web Apps, autenticándose con el token de despliegue del recurso. El primer comando lo obtiene en una variable, sin mostrarlo en pantalla; el segundo publica el contenido de dist en el ambiente de producción. Este despliegue manual sirve para la primera publicación; después lo automatiza el pipeline.

Powershell:

```
$token = az staticwebapp secrets list --name
swa-your_static_web_app_name --resource-group rg-cleanlink --query
"properties.apiKey" -o tsv

npx @azure/static-web-apps-cli deploy ./dist --deployment-token
$token --env production
```

Verificación de la aplicación en producción

Una vez publicado el frontend, se verifica el funcionamiento completo de la solución accediendo a la URL pública de Azure Static Web Apps. La aplicación consulta los

endpoints `/api/health` y `/api/db-health` del backend y muestra su estado. Que ambos aparezcan como "Conectada" confirma la comunicación entre el frontend, la API en Azure App Service y la base de datos en Azure SQL Database.



Figura 9. Aplicación CleanLink en el ambiente de producción, servida desde Azure Static Web Apps con la API y la base de datos reportadas como conectadas.

Habilitar la autenticación básica de publicación en App Service

El despliegue automatizado del backend con GitHub Actions utiliza el perfil de publicación de la aplicación, que depende de la autenticación básica del sitio. En suscripciones nuevas esta opción puede estar deshabilitada por defecto, y en ese caso el despliegue falla con el error "Publish profile is invalid for app-name and slot-name provided". Antes de obtener el perfil, se verifica.

PowerShell

```
az resource show --resource-group your_resource_group --name scm
--namespace Microsoft.Web --resource-type
basicPublishingCredentialsPolicies --parent sites/your_app_name
--query properties.allow
```

Si es necesario, se habilita la política de credenciales de publicación.

PowerShell

```
az resource update --resource-group your_resource_group --name scm
--namespace Microsoft.Web --resource-type
basicPublishingCredentialsPolicies --parent sites/your_app_name
--set properties.allow=true --query properties.allow
```

Configuración de credenciales de despliegue (GitHub Secrets)

Secreto	Servicio	Uso
AZURE_STATIC_WEB_APPS_API_TOKEN	Azure Static Web Apps (frontend)	Llave de despliegue para publicar los archivos compilados.
AZURE_WEBAPP_PUBLISH_PROFILE	Azure App Service (backend)	Perfil de publicación con las credenciales para enviar el código de la API.

Ambos valores se obtienen con Azure CLI y se copian al portapapeles sin mostrarse en pantalla:

Powershell

```
az staticwebapp secrets list --name swa-your_static_web_app_name
--resource-group rg-your_resource_group --query
"properties.apiKey" -o tsv | Set-Clipboard

az webapp deployment list-publishing-profiles --name your_app_name
--resource-group your_resource_group --xml | Set-Clipboard
```

Cada valor se pega en el campo Secret al crear el secreto correspondiente. Una vez guardado, GitHub no vuelve a mostrarlo: la interfaz solo lista el nombre y la fecha de actualización.

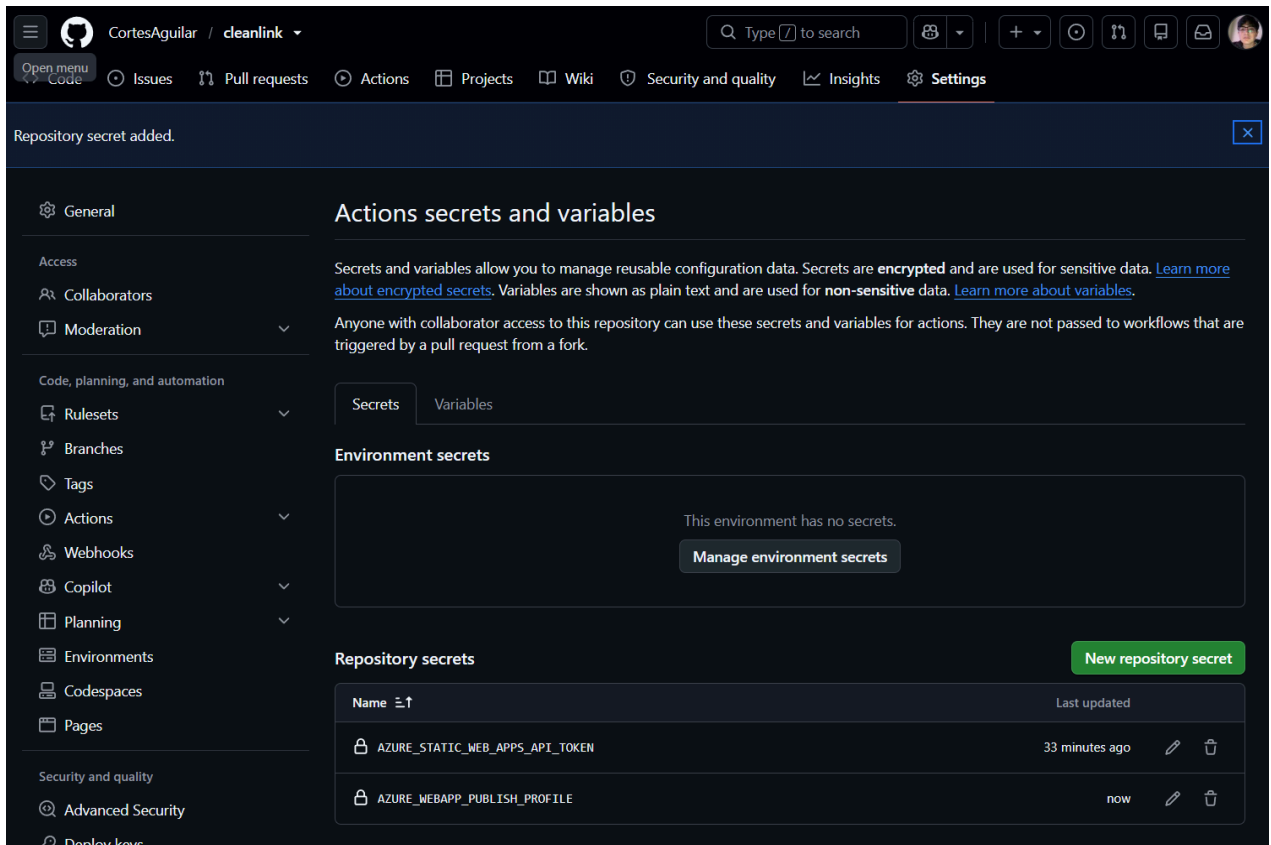


Figura 10. Secretos AZURE_STATIC_WEB_APPS_API_TOKEN y AZURE_WEBAPP_PUBLISH_PROFILE registrados en GitHub Actions; los valores permanecen cifrados y no son visibles.

Pipeline de CI/CD del frontend (GitHub Actions)

El despliegue del frontend se automatiza con un workflow de GitHub Actions, definido en un archivo YAML dentro del repositorio. Cada vez que se suben cambios a la rama main que afecten la carpeta frontend, el pipeline descarga el código, instala Node.js, compila la aplicación con Vite y publica el resultado en Azure Static Web Apps. Al compilar, se define VITE_API_URL con la dirección del backend de producción, y la credencial de despliegue se lee del secreto configurado en el paso anterior.

Archivo: .github/workflows/frontend.yml

```
name: Deploy frontend (Azure Static Web Apps)

on:
  push:
    branches: [main]
    paths:
      - 'frontend/**'
```

```

- '.github/workflows/frontend.yml'
workflow_dispatch:

jobs:
  build_and_deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el código
        uses: actions/checkout@v4

      - name: Instalar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Compilar el frontend
        working-directory: frontend
        env:
          VITE_API_URL: https://your_app_name.azurewebsites.net
        run: |
          npm ci
          npm run build

      - name: Publicar en Azure Static Web Apps
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${secrets.AZURE_STATIC_WEB_APPS_API_TOKEN}
          action: upload
          app_location: frontend/dist
          skip_app_build: true
          skip_api_build: true

```

Tras subir ambos archivos al repositorio, cada workflow se ejecuta automáticamente y su resultado puede consultarse en la pestaña Actions de GitHub. Una ejecución exitosa aparece marcada con una palomita verde, junto con el mensaje del commit que la disparó y su duración. A partir de este punto, cualquier cambio en frontend/ o backend/ que se suba a la rama main se despliega en Azure sin intervención manual.

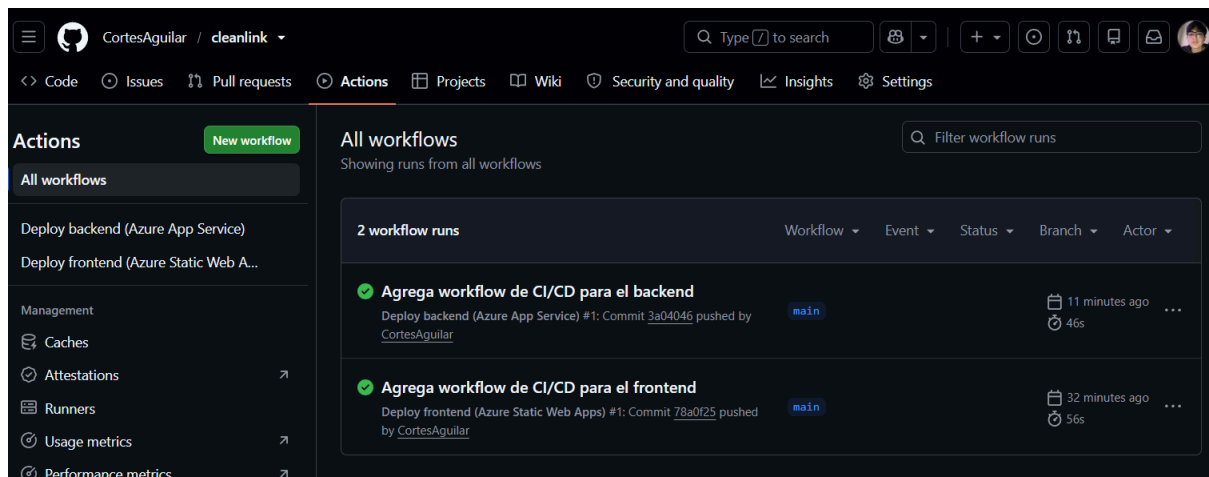


Figura 11. Ejecuciones exitosas de los workflows de despliegue del backend (Azure App Service) y del frontend (Azure Static Web Apps) en GitHub Actions.

Pipeline de CI/CD del backend (GitHub Actions)

El despliegue del backend se automatiza con un segundo workflow. Cada vez que se suben a la rama main cambios que afecten la carpeta backend, el pipeline descarga el código y lo publica en Azure App Service usando el perfil de publicación almacenado como secreto. A diferencia del frontend, este workflow no incluye un paso de compilación: el código se envía tal cual y es Azure quien instala las dependencias, gracias a la variable de aplicación SCM_DO_BUILD_DURING_DEPLOYMENT.

Archivo: .github/workflows/backend.yml

```
name: Deploy backend (Azure App Service)

on:
  push:
    branches: [main]
    paths:
      - 'backend/**'
      - '.github/workflows/backend.yml'
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el codigo
```

```
    uses: actions/checkout@v4

- name: Instalar Node.js
  uses: actions/setup-node@v4
  with:
    node-version: 24

- name: Publicar en Azure App Service
  uses: azure/webapps-deploy@v3
  with:
    app-name: your_app_name
    publish-profile: ${
secrets.AZURE_WEBAPP_PUBLISH_PROFILE }
    package: backend
```